MATEMATICAS III:
Tema I: Cálculo de raíces de ecuaciones no-lineales

Objetivo: Aproximar raíces de una ecuación no-lineal,

$$f(x) = 0$$

cuando no se puede (o no se quiere) efectuar el cálculo despejando de la ecuación. Estudiaremos algunos métodos que nos permitirán aproximar α, tal que $f(\alpha) = 0$.

1. Método de la Bisección

A partir de un intervalo inicial $[a_0, b_0]$, que contenga una única raiz, α, se construye

1. una sucesión de intervalos $[a_n, b_n]$
2. una sucesión de puntos m_n

mediante un proceso de subdivisión recursiva de manera que

- $\alpha \in [a_n, b_n] \quad \forall n$
- $m_n = \dfrac{a_n + b_n}{2}$
- $|\alpha - m_n| \leq \dfrac{|b_n - a_n|}{2} = \dfrac{b_0 - a_0}{2^{n+1}}$

El algoritmo, en pseudo-código, es como sigue,

```
Dados a_0, b_0,
    Para n ≥ 0
        m_n = 0,5 * (a_n + b_n)
        if (f(a_n) * f(m_n) < 0)
            b_{n+1} = m_n ; a_{n+1} = a_n
        elseif f(m_n) = 0 ≈ 0 break %% m es la raiz
        else
            a_{n+1} = m_n ; b_{n+1} = b_n
    end
```

El método de la bisección es siempre convergente si el intervalo inicial encierra una raíz. El criterio de parada puede hacerse sobre el número de iteraciones deseado o sobre una tolerancia pre-estabecida para el error. En el algoritmo anterior se cumple que

$$|\alpha - m_n| \leq \frac{b_n - a_n}{2}.$$

además, el proceso de subdivisión implica que

$$\frac{b_n - a_n}{2} = \frac{b_0 - a_0}{2^{n+1}},$$

por lo que el número de iteraciones necesario para asegurar que el error $|\alpha - m_n|$ es menor que una tolerancia pre-establecida se puede determinar a priori.

A continuación presentamos un código en Octave cuyo criterio de parada es el número de iteraciones, que se considera una entrada del código.

```
function [mm,cee]=bisec(a,b,ntimes)
for k=1:ntimes
    fa=fun(a);fb=fun(b); ce=.5*(b-a);
    m=a+ce; fm=fun(m);
    prod=fa*fm;
    if ( prod<0 )
       b=m; fb=fm;
    elseif (prod == 0)
       k=ntimes
    else
       a=m; fa=fm;
    end
    mm(k)=m; cee(k)=ce;
end
```

Observa que no se utilizan vectores para realizar el cálculo de las aproximaciones. Asegúrate de que entiendes el proceso de asignación de variables dentro del código, y que entiendes el paralelismo entre el pseudo-código anterior y esta función de Octave. Observa, además, que se calcula la cota para el error en cada iteración $0.5*(b_n - a_n)$ y se almacena en la variable `ce`. Como salidas, el programa produce un vector de aproximaciones, y un vector que contiene las cotas del error correspondiente a cada approximación.

Esta función requiere otra función auxiliar `fun()` que se encarga de evaluar la función $f(x)$. Para el caso en que

$$f(x) = x^3 + 5x + 1 \tag{1}$$

esta función se puede implementar como sigue:

```
function y=fun(x)
         y=x^3+5*x+1;
end
```

Ejercicio:

1. Cuenta el número de operaciones/evaluaciones de la función que se efectúan cuando Octave ejecuta esta función.
2. Piensa como podrias modificar el programa para asegurar que el error no sobrepase una tolerancia pre-establecida. La función resultante debería tener como entradas únicamente los puntos `a` y `b` y la tolerancia deseada.

La aplicación del código anterior para la función (1), cuya gráfica en el intervalo [-2,1] se puede ver en la figura 1, produce el siguiente resultado:

aproximaciones	cotas error
-0.5000	1.5000
0.2500	0.7500
-0.1250	0.3750
-0.3125	0.1875
-0.2188	0.0938
-0.1719	0.0469
-0.1953	0.0234
-0.2070	0.0117
-0.2012	0.0059
-0.1982	0.0029

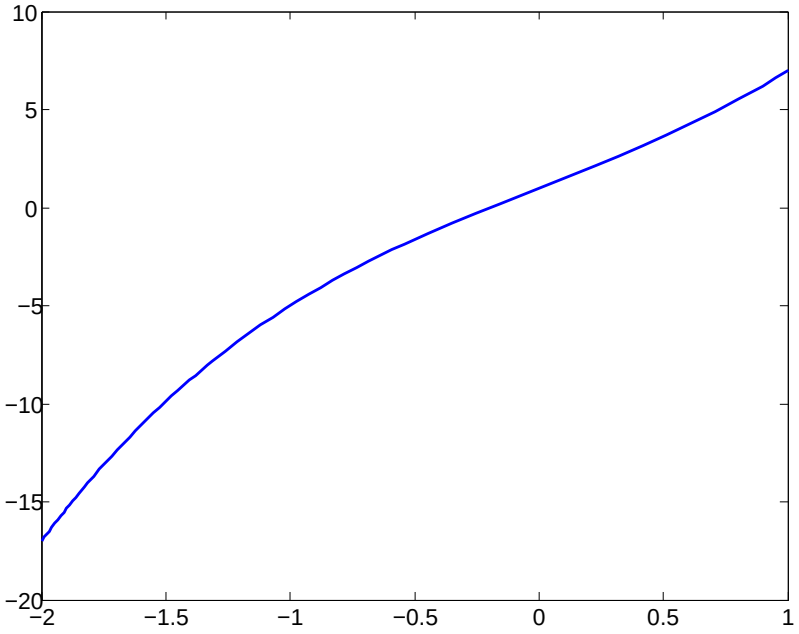

Figura 1: Gráfica de $f(x) = x^3 + 5x + 1$ en el intervalo [-2,1]

2. Método de la Regla Falsa

Funciona como el método de la bisección, pero la aproximación m_n a la raíz se calcula como el punto de corte con el eje de abcisas de la recta secante a la curva $f(x)$ en los puntos $(a_n, f(a_n))$, $(b_n, f(b_n))$.

Recta que pasa por $(a_n, f(a_n))$, $(b_n, f(b_n))$:

$$\frac{y - f(a_n)}{f(b_n) - f(a_n)} = \frac{x - a_n}{b_n - a_n}$$

Corte con $y = 0$

$$\frac{-f(a_n)}{f(b_n) - f(a_n)} = \frac{x - a_n}{b_n - a_n} \quad \rightarrow \quad x = a_n - f(a_n)\frac{b_n - a_n}{f(b_n) - f(a_n)}$$

El método de la regla falsa es siempre convergente y, en general, funciona mejor que el método de la bisección, pero puede ser llegar a ser tan lento como éste.

Ejercicio

Modifica las funciones de Octave del apartado anterior para implementar el método de la regla falsa. Ten en cuenta que ahora no tiene sentido calcular cotas del error como en el método de la bisección.

1. Confecciona el programa de manera que las entradas sean a_0, b_0 y el número de iteraciones que se desea realizar
2. Modifica el programa de manera que las entradas sean a_0, b_0 y una tolerancia prefijada.

3. Método de Newton

Este método utiliza un único punto de partida, y calcula la sucesión de aproximaciones a partir de la recta tangente a la curva $f(x)$ en cada una de las aproximaciones calculadas. Recordamos que la recta tangente a la curva $f(x)$ en el punto $(a, f(a))$ es

$$y - f(a) = f'(a)(x - a),$$

de manera que el corte de esta recta con el eje de abcisas ($y = 0$) se calcula como

$$0 - f(a) = f'(a)(x - a) \quad \rightarrow \quad x = a - f(a)/f'(a).$$

Para utilizar este método es necesario poder calcular la derivada de la función $f(x)$.

Algoritmo en pseudo-código:

```
Dado a_0
    Para n > 0
        a_{n+1} = a_n - f(a_n)/f'(a_n);
    end
```

Criterios de parada: Al contrario de lo que sucede en el método de la bisección, no hay garantias de que el método de Newton converja. Lo hace si el punto inicial está 'suficientemente cerca' de la raiz, o si la función cumple determinadas propiedades en el intervalo comprendido entre el punto de comienzo y la raiz. Si el método de Newton converge, entonces llega un momento en que

$$\text{Error}_{n+1} = |\alpha - a_{n+1}| \approx |\alpha - a_n|^2 = \text{Error}_n^2$$

Esto quiere decir que cuando el error comienza a hacerse pequeño, se reduce muy rapidamente. Asi, si

$$E_n \approx 10^{-1} \quad \rightarrow \quad E_{n+1} \approx 10^{-2} \quad \rightarrow \quad E_{n+2} \approx 10^{-4} \quad \rightarrow \quad E_{n+3} \approx 10^{-8} \quad \rightarrow \quad E_{n+4} \approx 10^{-16}$$

El método de Newton puede no converger, y hay que tener cuidado con la elección del punto inicial. Puesto que la convergencia hacia la raiz se produce 'ganando dígitos significativos', se suele implementar como criterio de parada para el método de Newton el hecho de que

$$|a_{n+1} - a_n| = \left|\frac{f(a_n)}{f'(a_n)}\right| \leq \text{xtol},$$

donde 'xtol' es una tolerancia prefijada de antemano.

El siguiente programa de Octave implementa el método de Newton, con criterio de parada basado en una tolerancia *xtol* y un número máximo de iteraciones, `nmax`. Observa que necesita de **dos** funciones auxiliares, `fun()` y`funp()`, que evalúan la función y su derivada.

```
function [mm,iter,erest]=newton(a,xtol)
    nmax=10;
    delta=1;   %% almacena el incremento a_{n+1} - a_n
    k=0;  %% contador de iteraciones
```

```
while (abs(delta)>xtol)& (k <= nmax)
      fa=fun(a);fpa=funp(a);
      delta= fa/fpa;
      a=a-delta;
      k=k+1:
      mm(k)=a;
end
iter=k;
erest=abs(delta);
```

Asegúrate de que entiendes cómo funcionan todas y cada una de las líneas del código anterior. Escribe el código Octave para la función `funp()` correspondiente a la función $f(x)$ de (1)